NISHANTHAN V
241801190

EXPERIMENT 6a - EXPLORATORY DATA ANALYSIS

Learning Objective

The objective of this experiment is to understand and perform Exploratory Data Analysis (EDA) on a real-world Netflix Titles dataset using Python. The experiment involves exploring Netflix movies and TV shows, analysing content distribution, ratings, release and addition trends, duration, countries, genres, directors, actors, and textual descriptions using suitable statistical techniques and visualizations.

Libraries Used / Purpose

Pandas – Used for loading, cleaning, manipulating, and analysing the Netflix dataset.

NumPy – Used for numerical and statistical operations.

Matplotlib – Used to create charts and visualizations.

Seaborn – Used to create statistical plots and improve data visualization.

WordCloud – Used to generate a word cloud from Netflix descriptions.

Collections.Counter – Used to count frequent genres, actors, directors, and words.

Re – Used for text cleaning and preprocessing.

KaggleHub – Used to download the Netflix Titles dataset from Kaggle.

Program Code

```
!pip install -q kagglehub wordcloud

import kagglehub import
pandas as pd import numpy
as np import
matplotlib.pyplot as plt import
seaborn as sns import os
import re
from collections import Counter from
wordcloud import WordCloud
```

```
sns.set_theme(style="whitegrid")

path = kagglehub.dataset_download("shivamb/netflix-shows")

print("Dataset path:", path)

files = os.listdir(path) print("Files:",
files)

csv_file = [
os.path.join(path, file)
for file in files     if
file.endswith(".csv")
][0]

df = pd.read_csv(csv_file)

print("\n--- First 5 Records ---")
display(df.head())

print("\n--- Dataset Shape ---") print(df.shape)

print("\n--- Column Names ---")
print(df.columns.tolist())

print("\n--- Dataset Information ---") df.info()

print("\n--- Missing Values ---")
display(df.isnull().sum())

print("\n--- Duplicate Records ---")
print(df.duplicated().sum())

df_clean = df.copy()

df_clean['date_added'] = pd.to_datetime(
df_clean['date_added'],     errors='coerce'
)

df_clean['release_year'] = pd.to_numeric(
df_clean['release_year'],     errors='coerce'
)
```

```
df_clean['duration_num'] = pd.to_numeric(
df_clean['duration'].str.extract(r'(\d+)')[0],
errors='coerce'
)

df_clean['duration_unit'] = (
df_clean['duration']
    .str.extract(r'([A-Za-z]+)')[0]
)

print("\n--- Cleaned Dataset ---")
display(df_clean.head())

print("\n" + "=" * 80)
print("1. CONTENT DISTRIBUTION")
print("=" * 80)

content_counts = df_clean['type'].value_counts()

print("\n--- Movies vs TV Shows ---")
display(content_counts.reset_index().rename(
columns={
        'type': 'Content Type',
        'count': 'Number of Titles'
    }
))

plt.figure(figsize=(7, 5))
sns.countplot(
data=df_clean,     x='type',
order=content_counts.index
)
plt.title('Movies vs TV Shows')
plt.xlabel('Content Type')
plt.ylabel('Number of Titles') plt.show()
print("\n--- Rating Distribution ---")

rating_counts = df_clean['rating'].value_counts()

display(
rating_counts.reset_index().rename(
columns={
            'rating': 'Rating',
            'count': 'Number of Titles'
```

```
        }
    )
)

plt.figure(figsize=(12, 6))
sns.countplot(
data=df_clean,     y='rating',
    order=rating_counts.head(15).index
)
plt.title('Netflix Rating Distribution')
plt.xlabel('Number of Titles')
plt.ylabel('Rating') plt.show()

print("\n--- Countries with Most Netflix Content ---")

country_series = (
df_clean['country']     .dropna()
    .str.split(',')
    .explode()
    .str.strip()
)

country_counts = country_series.value_counts()

display(
    country_counts.head(10)
    .reset_index()
.rename(
columns={
            'country': 'Country',
            'count': 'Number of Titles'
        }
    )
)

plt.figure(figsize=(10, 6)) sns.barplot(
x=country_counts.head(10).values,
y=country_counts.head(10).index
)
plt.title('Top 10 Countries by Netflix Content')
plt.xlabel('Number of Titles')
plt.ylabel('Country') plt.show()

print("\n" + "=" * 80)
```

```
print("2. TIME-BASED TREND ANALYSIS")
print("=" * 80)

release_counts = df_clean['release_year'].value_counts().sort_index()

peak_release_years = (
df_clean['release_year']
    .value_counts()
    .head(10)
)

print("\n--- Years with Most Releases ---")
display(
   peak_release_years
   .reset_index()
.rename(
columns={
          'release_year': 'Release Year',
          'count': 'Number of Titles'
       }
   )
)

plt.figure(figsize=(12, 6))
plt.plot(
release_counts.index,
release_counts.values,
marker='o'
)
plt.title('Netflix Titles by Release Year')
plt.xlabel('Release Year')
plt.ylabel('Number of Titles') plt.show()

added_year_counts = (
df_clean['date_added']
    .dt.year
    .value_counts()
    .sort_index()
)

print("\n--- Titles Added to Netflix by Year ---")
display(     added_year_counts
```

```
    .reset_index()
.rename(
columns={
            'date_added': 'Year Added',
            'count': 'Number of Titles'
        }
    )
)

plt.figure(figsize=(12, 6)) plt.plot(
    added_year_counts.index,
added_year_counts.values,      marker='o'
)
plt.title('Netflix Content Added by Year')
plt.xlabel('Year') plt.ylabel('Number of
Titles Added') plt.show()

content_growth = (
added_year_counts
    .cumsum()
)

plt.figure(figsize=(12, 6)) plt.plot(
    content_growth.index,
    content_growth.values,      marker='o'
)
plt.title('Cumulative Netflix Content Growth')
plt.xlabel('Year') plt.ylabel('Cumulative
Number of Titles') plt.show()

print("\n" + "=" * 80)
print("3. DURATION AND CONTENT TYPE ANALYSIS") print("="
* 80)

movie_duration = df_clean[
    (df_clean['type'] == 'Movie') &
    (df_clean['duration_unit'].str.lower() == 'min')
]['duration_num'].dropna()

print("\n--- Movie Duration Statistics ---")
display(movie_duration.describe())

plt.figure(figsize=(10, 6))
sns.histplot(
```

```
movie_duration,
bins=30,     kde=True
)
plt.title('Distribution of Movie Durations')
plt.xlabel('Duration (Minutes)')
plt.ylabel('Number of Movies') plt.show()

tv_seasons = df_clean[
df_clean['type'] == 'TV Show'
]['duration_num'].dropna()

season_counts = tv_seasons.value_counts().sort_index()

print("\n--- TV Shows by Number of Seasons ---")
display(     season_counts     .reset_index()
.rename(         columns={
           'duration_num': 'Number of Seasons',
           'count': 'Number of TV Shows'
       }
    )
)

plt.figure(figsize=(10, 6)) sns.barplot(
x=season_counts.index.astype(str),
y=season_counts.values
)
plt.title('TV Shows by Number of Seasons')
plt.xlabel('Number of Seasons')
plt.ylabel('Number of TV Shows') plt.show()

print("\n" + "=" * 80)
print("4. GENRE AND CATEGORY ANALYSIS")
print("=" * 80)

genre_series = (
df_clean['listed_in']     .dropna()
    .str.split(',')
    .explode()
    .str.strip()
)

genre_counts = genre_series.value_counts()
```

```
print("\n--- Top 10 Genres ---")
display(
genre_counts.head(10)
    .reset_index()
.rename(
columns={
            'listed_in': 'Genre',
            'count': 'Number of Titles'
        }
    )
)

plt.figure(figsize=(10, 6)) sns.barplot(
    x=genre_counts.head(10).values,
    y=genre_counts.head(10).index
) plt.title('Top 10 Netflix
Genres') plt.xlabel('Number of
Titles') plt.ylabel('Genre')
plt.show()

director_series = (
df_clean['director']     .dropna()
    .str.split(',')
    .explode()
    .str.strip()
)

director_counts = director_series.value_counts()

print("\n--- Directors with Most Titles ---")
display(
    director_counts.head(10)
    .reset_index()
.rename(
columns={
            'director': 'Director',
            'count': 'Number of Titles'
        }
    )
)

plt.figure(figsize=(10, 6)) sns.barplot(
x=director_counts.head(10).values,
y=director_counts.head(10).index
```

```
)
plt.title('Top 10 Directors by Number of Titles')
plt.xlabel('Number of Titles')
plt.ylabel('Director') plt.show()

actor_series = (
df_clean['cast']     .dropna()
    .str.split(',')
    .explode()
    .str.strip()
)

actor_counts = actor_series.value_counts()

print("\n--- Actors Appearing Most Frequently ---")
display(     actor_counts.head(10)
    .reset_index()
.rename(
columns={
            'cast': 'Actor',
            'count': 'Number of Titles'
        }
    )
)

plt.figure(figsize=(10, 6)) sns.barplot(
x=actor_counts.head(10).values,
y=actor_counts.head(10).index
)
plt.title('Top 10 Actors by Number of Titles')
plt.xlabel('Number of Titles')
plt.ylabel('Actor') plt.show()

print("\n" + "=" * 80)
print("5. TEXT AND DESCRIPTION
ANALYSIS")
print("=" * 80)

descriptions = (
df_clean['description']
    .dropna()
    .astype(str)
    .str.lower()
    .str.replace(r'[^a-z\s]', ' ', regex=True)
```

```
)

stop_words = {
    'the', 'and', 'a', 'of', 'to', 'in', 'is', 'on',
    'for', 'with', 'his', 'her', 'their', 'from',
    'as', 'by', 'an', 'at', 'this', 'that', 'into',
    'after', 'when', 'who', 'has', 'have', 'be',
    'are', 'it', 'its', 'they', 'will', 'but'
}

words = []

for text in descriptions:     words.extend(
word for word in text.split()         if word not in
stop_words and len(word) > 2
    )

word_counts = Counter(words)

print("\n--- Most Common Words in Descriptions ---")

display(     pd.DataFrame(
word_counts.most_common(20),
columns=['Word', 'Frequency']
    )
)

wordcloud = WordCloud(
width=1200,     height=600,
background_color='white'
).generate(' '.join(words))

plt.figure(figsize=(14, 7))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off') plt.title('Word Cloud of Netflix
Descriptions') plt.show()

print("\n" + "=" * 80)
print("6. MONTHS WITH MOST CONTENT ADDED")
print("=" * 80)

month_counts = (
df_clean['date_added']
    .dropna()
```

```
    .dt.month
    .value_counts()
    .sort_index()
)
month_names = {
    1: 'January',
    2: 'February',
    3: 'March',
    4: 'April',
    5: 'May',
    6: 'June',
    7: 'July',
    8: 'August',
    9: 'September',
    10: 'October',
    11: 'November',
    12: 'December'
}

month_df = pd.DataFrame({
    'Month': [
month_names[m]         for m in
month_counts.index
    ],
    'Number of Titles Added': month_counts.values
})

display(month_df)

plt.figure(figsize=(12, 6))
sns.barplot(
data=month_df,
x='Month',
    y='Number of Titles Added'
)
plt.title('Netflix Content Added by Month')
plt.xlabel('Month') plt.ylabel('Number of
Titles Added') plt.xticks(rotation=45)
plt.show()

print("\n" + "=" * 80)
print("7. COUNTRIES PRODUCING MOVIES VS TV SHOWS") print("="
* 80)
```

```
country_type = df_clean[
    ['country', 'type']
].dropna().copy()

country_type['country'] = (
country_type['country']
    .str.split(',')
)

country_type = country_type.explode('country')

country_type['country'] = (
country_type['country']
    .str.strip()
)

country_type_counts = pd.crosstab(
country_type['country'],     country_type['type']
)

country_type_counts['Total'] = (
country_type_counts.sum(axis=1)
)

country_type_top = (
country_type_counts
    .sort_values('Total', ascending=False)
    .head(10)
)

print("\n--- Countries Producing Movies vs TV Shows ---") display(country_type_top)

country_type_top[['Movie', 'TV Show']].plot(
kind='bar',     figsize=(12, 6)
)

plt.title('Movies vs TV Shows by Country')
plt.xlabel('Country') plt.ylabel('Number of
Titles') plt.xticks(rotation=45) plt.show()

print("\n" + "=" * 80)
print("8. COMMON RATINGS FOR MOVIES VS TV SHOWS") print("="
* 80)
```

```
rating_type = pd.crosstab(
df_clean['rating'],     df_clean['type']
)

rating_type['Total'] = rating_type.sum(axis=1)

print("\n--- Ratings by Content Type ---") display(
    rating_type.sort_values(
        'Total',
        ascending=False
    ).head(15)
)

movie_ratings = (
    df_clean[df_clean['type'] == 'Movie']['rating']
    .value_counts()
    .head(10)
)

tv_ratings = (     df_clean[df_clean['type'] == 'TV
Show']['rating']
    .value_counts()
    .head(10)
)

print("\n--- Most Common Movie Ratings ---")
display(movie_ratings.reset_index().rename(
columns={
        'rating': 'Rating',
        'count': 'Number of Movies'
    }
))

print("\n--- Most Common TV Show Ratings ---")
display(tv_ratings.reset_index().rename(
columns={
        'rating': 'Rating',
        'count': 'Number of TV Shows'
    }
))

print("\n" + "=" * 80)
print("9. OLDEST AND NEWEST TITLES")
print("=" * 80)
```

```
oldest_titles = df_clean[
df_clean['release_year'] ==
df_clean['release_year'].min()
]][['title', 'type', 'release_year']]

newest_titles = df_clean[
df_clean['release_year'] ==
df_clean['release_year'].max()
]][['title', 'type', 'release_year']]

print("\n--- Oldest Titles ---")
display(oldest_titles)

print("\n--- Newest Titles ---")
display(newest_titles)

print("\n" + "=" * 80)
print("10. GENRE CO-OCCURRENCE ANALYSIS")
print("=" * 80)

genre_pairs = Counter()

for genres in df_clean['listed_in'].dropna():

    genre_list = [
genre.strip()        for genre in
genres.split(',')
    ]

    for i in range(len(genre_list)):

        for j in range(i + 1, len(genre_list)):
            pair =
tuple(
sorted(
                    [genre_list[i], genre_list[j]]
                )
            )

            genre_pairs[pair] += 1

genre_pair_df = pd.DataFrame(
genre_pairs.most_common(15),
```

```
    columns=['Genre Pair', 'Co-occurrence Frequency']
)

genre_pair_df[     ['Genre 1', 'Genre
2'] ] = pd.DataFrame(
genre_pair_df['Genre Pair'].tolist(),
index=genre_pair_df.index
)

genre_pair_df = genre_pair_df[
    [
        'Genre 1',
        'Genre 2',
        'Co-occurrence Frequency'
    ]
]

print("\n--- Most Frequently Co-occurring Genres ---") display(genre_pair_df)

plt.figure(figsize=(12, 7))

sns.barplot(     data=genre_pair_df,     x='Co-occurrence
Frequency',     y=genre_pair_df['Genre 1'] + ' + ' +
genre_pair_df['Genre 2']
)

plt.title('Top Genre Co-occurrences')
plt.xlabel('Co-occurrence Frequency')
plt.ylabel('Genre Pair') plt.show()

print("\n" + "=" * 80)
print("EDA SUMMARY")
print("=" * 80)

print("Total titles:", len(df_clean))
print("Movies:", (df_clean['type'] == 'Movie').sum()) print("TV
Shows:", (df_clean['type'] == 'TV Show').sum()) print("Most
common rating:", df_clean['rating'].mode()[0]) print("Most
common genre:", genre_counts.index[0]) print("Country with
most content:", country_counts.index[0]) print("Oldest release
year:", int(df_clean['release_year'].min())) print("Newest release
year:", int(df_clean['release_year'].max()))
```

Output

After completing this experiment, students will be able to preprocess and clean customer review data using Pandas, apply spaCy NLP pipelines to unstructured text, perform tokenization and lemmatization, identify and analyze Part-of-Speech tags, calculate the frequency of major POS categories, perform Named Entity Recognition, extract meaningful entities such as PERSON, ORG, DATE, MONEY, QUANTITY, and PRODUCT when recognized by the selected spaCy model, develop a reusable information extraction function, and organize extracted linguistic information into structured formats for further customer feedback analysis.

--- Dataset Shape ---
(8807, 12)

--- Column Names ---
['show_id', 'type', 'title', 'director', 'cast', 'country',
 'date_added', 'release_year', 'rating', 'duration',
 'listed_in', 'description']

--- Duplicate Records ---
0

Learning Outcome

After completing this experiment, students will be able to perform Exploratory Data Analysis on a real-world Netflix dataset, explore and clean structured data, analyse movie and TV show distributions, investigate ratings and country-wise content production, identify release and Netflix addition trends, analyse movie durations and TV show seasons, determine popular genres, directors, and actors, perform textual analysis using word frequency and word clouds, compare ratings and content types across countries, identify the oldest and newest titles, analyse genre co-occurrence patterns, and communicate meaningful insights from data using appropriate statistical techniques and visualizations.